Escuela Superior Politécnica del Litoral

Facultad de Ingeniería en Electricidad y Computación

Ing. En Ciencias de la Computación

PAO II 2026-2027

Curso: Diseño de software – Paralelo 3

Taller 5: Creacionales

Integrantes:

Alex Moreno R.

Wilson Chang

Joel Claudett

Jorge Martínez



Tabla de contenido.

1. **Reporte diseño de patrones creacionales aplicados al sistema propuesto**
2. **Diagrama de clases UML**
3. **Diagrama de secuencias UML**

Reporte patrones de diseño creacionales aplicados al sistema propuesto:

Para este sistema se identifican dos patrones de diseño aplicables al sistema: Singleton y Factory Method.

Patrón Singleton (Gestor de Conexión y Acceso a Datos)
Objetivo: Garantiza que la clase encargada a la conexión de la base de datos tenga una única instancia en toda la aplicación y proporciona un punto de acceso global a ella.

Motivación: El sistema requiere un componente centralizado que gestione la conexión con la base de datos y atienda todas las consultas y actualizaciones. Permitir múltiples instancias de este componente podría duplicar recursos, saturar las conexiones y generar conflictos de concurrencia al modificar inventarios y precios. Singleton restringe la instanciación de la clase a un solo objeto y centraliza el control de acceso.

Consecuencias:

Ventajas: Control estricto sobre el acceso a la base de datos central, ahorro de recursos del sistema y garantía de consistencia en el estado de la conexión.

Desventajas: Puede complicar las pruebas unitarias (dificultad para sustituir con objetos simulados si no se abstrae mediante interfaces) y hay riesgo de acoplamiento si se utiliza como variable global

Relación con SOLID:

Se alinea con el SRP al delegar exclusivamente la gestión de la infraestructura de datos a una sola clase.

Al combinarse con una interfaz de repositorio, apoya el DIP al desacoplar las clases de negocio de la implementación concreta de la conexión.

Supuestos: Se asume que el acceso a la base de datos maneja su propia sincronización de hilos (thread-safe) a nivel de la plataforma Java.

Patrón Factory Method (Creación Dinámica de Productos)
Objetivo: Define una interfaz para crear un objeto, pero dejamos que las subclases (o una jerarquía de fábricas) decidan qué clase instanciar, permitiendo que la creación se delegue en tiempo de ejecución.

Motivación: El sistema maneja diferentes variantes de productos con atributos específicos (los productos físicos requieren peso y dimensiones; productos digitales, tamaño y formato). Además, el sistema debe ser capaz de soportar nuevos tipos de productos en el futuro sin modificar el código cliente existente (cumpliendo con la extensibilidad). Factory method desacopla el código cliente de las clases concretas de productos mediante una interfaz común.

Consecuencias:

Ventajas: Cumplimiento del OCP (se pueden añadir nuevos tipos de productos creando nuevas subclases de producto y fábrica sin tocar el código existente) y eliminación de acoplamientos rígidos con operadores switch o if-else gigantes en el código cliente.

Desventajas: Incrementa la complejidad estructural del código al requerir nuevas clases e interfaces paralelas por cada tipo de producto introducido.

Relación con SOLID:

Cumple plenamente con el **Principio Abierto/Cerrado (OCP)** y el **Principio de Responsabilidad Única (SRP)**, ya que la lógica de creación de objetos se aísla en fábricas dedicadas en lugar de contaminar la lógica de negocio de la tienda.

Supuestos: Se asume que los identificadores de tipos de productos (ej. "fisico", "digital") se reciben como parámetros desde la interfaz de usuario o capa de servicio para que la fábrica decida la instanciación dinámica.

Diagrama de Clases UML (Estructura por Paquetes)

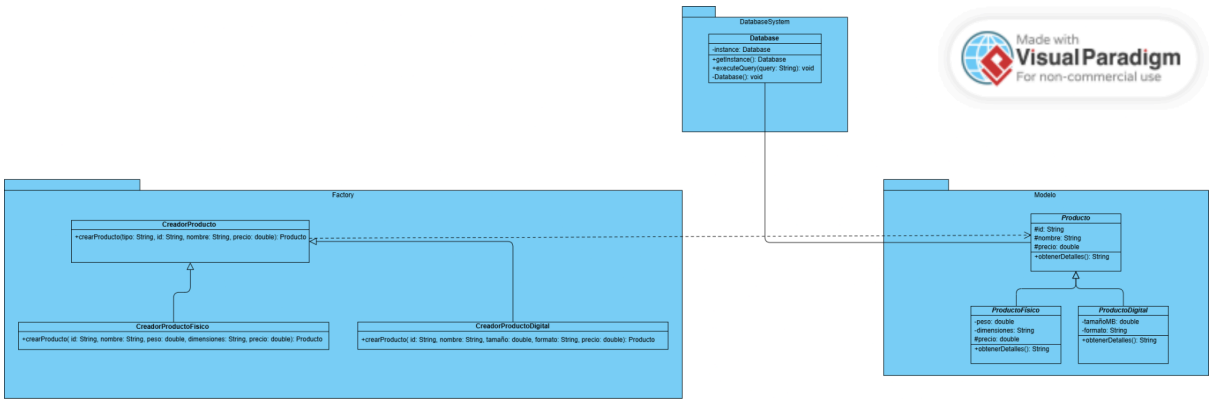


Diagrama de Secuencia: Patrón Singleton (Gestión de BD)

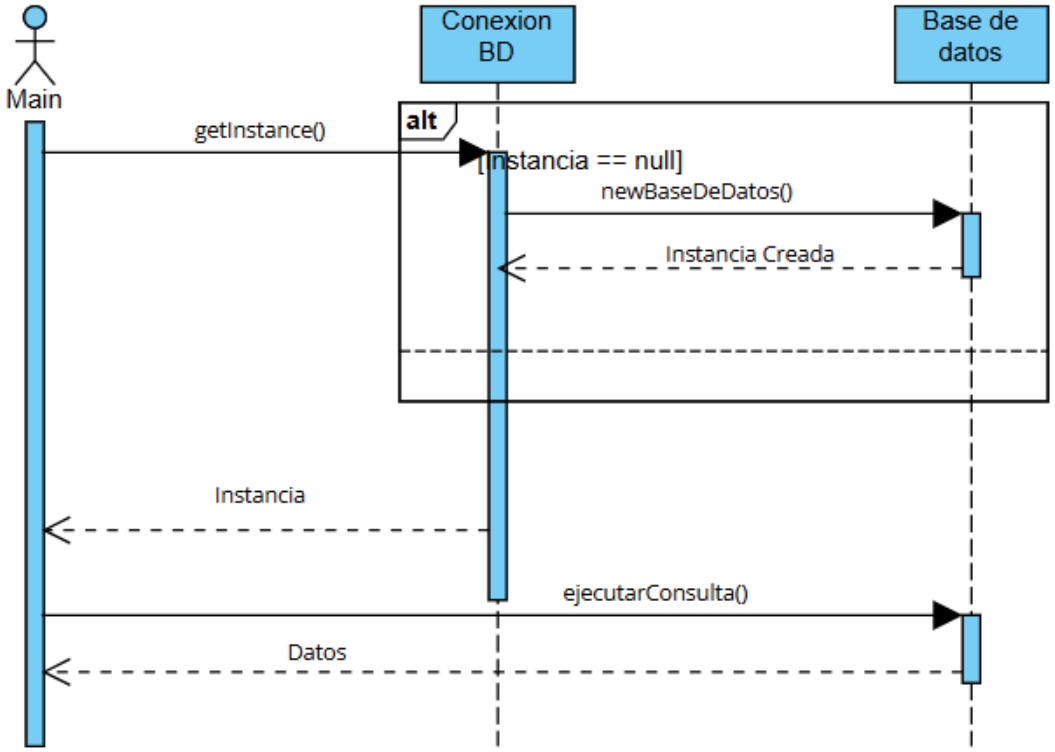


Diagrama de Secuencia: Patrón Factory Method (Creación de Productos)

